03MIAR - Algoritmos de optimización

Actividad guiada 3

Lucas Tamara Trejo

Repositorio con el notebook y el PDF: https://github.com/lucastamaraucv/AG3

Presentación

Profesor:

- Grupos A2, A3, B1, B2: David Álvarez Robles (dalvarezr@professor.universidadviu.com)

Objetivos

- Comprender el uso de librerías de simulación del problema del agente viajero en Python
- Implementar y visualizar algoritmos de búsqueda basados en metaheurísticas
- Algoritmo de búsqueda aleatoria
- Algoritmo de búsqueda local
- Algoritmo Simulated Annealing

Librerías y simulación del problema del agente viajero en Python

- Uso de librería tsplib95
- Visualización de datos

In [1]:
```
!pip install tsplib95
import tsplib95
import random
from math import e
import math
import copy
import urllib.request
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: tsplib95 in c:\users\lucas tamara\appdata\roaming
\python\python313\site-packages (0.7.1)
Requirement already satisfied: Click>=6.0 in c:\programdata\anaconda3\lib\site-pa
ckages (from tsplib95) (8.2.1)
Requirement already satisfied: Deprecated~=1.2.9 in c:\users\lucas tamara\appdata
\roaming\python\python313\site-packages (from tsplib95) (1.2.18)
Requirement already satisfied: networkx~=2.1 in c:\users\lucas tamara\appdata\roa
ming\python\python313\site-packages (from tsplib95) (2.8.8)
Requirement already satisfied: tabulate~=0.8.7 in c:\users\lucas tamara\appdata\r
oaming\python\python313\site-packages (from tsplib95) (0.8.10)
Requirement already satisfied: wrapt<2,>=1.10 in c:\programdata\anaconda3\lib\sit
e-packages (from Deprecated~=1.2.9->tsplib95) (1.17.0)
Requirement already satisfied: colorama in c:\programdata\anaconda3\lib\site-pack
ages (from Click>=6.0->tsplib95) (0.4.6)
```

In [2]:
```
# Descargamos el fichero de datos (Matriz de distancias)
file = "swiss42.tsp" ;
urllib.request.urlretrieve("https://raw.githubusercontent.com/mastqe/tsplib/refs

#urllib.request.urlretrieve("https://raw.githubusercontent.com/mastqe/tsplib/ref
#!gzip -d swiss42.tsp.gz     #Descomprimir el fichero de datos

#Coordendas 51-city problem (Christofides/Eilon)
#file = "eil51.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelbe

#Coordenadas - 48 capitals of the US (Padberg/Rinaldi)
#file = "att48.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelbe


#http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95
#Documentacion :
  # http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf
  # https://tsplib95.readthedocs.io/en/stable/pages/usage.html
  # https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
  # https://pypi.org/project/tsplib95/
```

Out[2]:
```
('swiss42.tsp', <http.client.HTTPMessage at 0x24d0e15d810>)
```

In [3]:
```
problema = tsplib95.load(file)
nodos = list(problema.get_nodes())
aristas = list(problema.get_edges())

print(problema) # Descripción estándar
print(nodos) # Nodos 0, 1, 2 ... 42
print(aristas) # Ver https://github.com/mastqe/tsplib/blob/master/swiss42.tsp
```

```
NAME: swiss42
COMMENT: 42 Staedte Schweiz (Fricker)
TYPE: TSP
DIMENSION: 42
EDGE_WEIGHT_TYPE: EXPLICIT
EDGE_WEIGHT_FORMAT: FULL_MATRIX
EDGE_WEIGHT_SECTION:
0 15 30 23 32 55 33 37 92 114 92 110 96 90 74 76 82 67 72 78 82 159 122 131 206 1
12 57 28 43 70 65 66 37 103 84 125 129 72 126 141 183 124
15 0 34 23 27 40 19 32 93 117 88 100 87 75 63 67 71 69 62 63 96 164 132 131 212 1
06 44 33 51 77 75 72 52 118 99 132 132 67 139 148 186 122
30 34 0 11 18 57 36 65 62 84 64 89 76 93 95 100 104 98 57 88 99 130 100 101 179 8
6 51 4 18 43 45 95 45 115 93 152 159 100 112 114 153 94
23 23 11 0 11 48 26 54 70 94 69 89 75 84 84 89 92 89 54 78 99 141 111 109 190 89
44 11 29 54 56 89 47 118 96 147 151 90 122 126 163 101
32 27 18 11 0 40 20 58 67 92 61 78 65 76 83 89 91 95 43 72 110 141 116 105 190 81
34 19 35 57 63 97 58 129 107 156 158 92 129 127 161 95
55 40 57 48 40 0 23 55 96 123 78 75 62 36 56 66 63 95 37 34 137 174 156 129 224 9
0 15 59 75 96 103 105 91 158 139 164 156 78 169 163 191 115
33 19 36 26 20 23 0 45 85 111 75 82 69 60 63 70 71 85 44 52 115 161 136 122 210 9
1 25 37 54 78 81 90 68 136 116 150 147 76 148 147 180 111
37 32 65 54 58 55 45 0 124 149 118 126 113 80 42 42 49 40 87 60 94 195 158 163 24
2 135 65 63 79 106 101 50 66 118 104 109 103 36 160 178 218 153
92 93 62 70 67 96 85 124 0 28 29 68 63 122 148 155 156 159 67 129 148 78 80 39 12
9 46 82 65 55 40 61 157 97 159 135 212 221 159 110 72 95 35
114 117 84 94 92 123 111 149 28 0 54 91 88 150 174 181 182 181 95 157 159 50 65 2
7 102 65 110 87 73 50 68 176 112 166 142 229 241 184 99 46 69 38
92 88 64 69 61 78 75 118 29 54 0 39 34 99 134 142 141 157 44 110 161 103 109 52 1
54 22 63 68 66 61 81 158 107 175 151 216 219 150 137 100 115 37
110 100 89 89 78 75 82 126 68 91 39 0 14 80 129 139 135 167 39 98 187 136 148 81
186 28 61 92 97 98 117 173 134 204 181 232 229 153 176 137 143 62
96 87 76 75 65 62 69 113 63 88 34 14 0 72 117 128 124 153 26 88 174 136 142 82 18
7 32 48 79 85 89 106 159 121 191 168 219 216 140 168 134 145 64
90 75 93 84 76 36 60 80 122 150 99 80 72 0 59 71 63 116 56 25 170 201 189 151 252
104 44 95 111 130 138 130 127 192 174 186 172 90 205 193 214 135
74 63 95 84 83 56 63 42 148 174 134 129 117 59 0 11 8 63 93 35 135 223 195 184 27
3 146 71 95 113 138 138 81 107 159 146 132 113 32 200 209 243 171
76 67 100 89 89 66 70 42 155 181 142 139 128 71 11 0 11 54 103 46 130 230 198 192
279 155 80 99 117 143 141 74 107 155 143 122 102 22 202 215 250 179
82 71 104 92 91 63 71 49 156 182 141 135 124 63 8 11 0 65 100 39 140 232 203 192
281 153 78 103 121 147 146 85 115 164 152 133 112 33 208 218 251 178
67 69 98 89 95 95 85 40 159 181 157 167 153 116 63 54 65 0 127 92 83 224 180 199
269 175 106 95 109 135 125 21 80 107 100 71 63 33 173 205 249 191
72 62 57 54 43 37 44 87 67 95 44 39 26 56 93 103 100 127 0 67 153 145 139 96 196
53 23 60 70 81 95 134 101 172 149 194 190 115 160 138 159 80
78 63 88 78 72 34 52 60 129 157 110 98 88 25 35 46 39 92 67 0 152 207 188 162 258
119 48 89 107 129 134 108 114 176 159 163 147 66 200 197 224 147
82 96 99 99 110 137 115 94 148 159 161 187 174 170 135 130 140 83 153 152 0 188 1
28 184 222 183 139 95 95 110 91 62 54 24 23 81 110 113 108 164 217 184
159 164 130 141 141 174 161 195 78 50 103 136 136 201 223 230 232 224 145 207 188
0 65 57 51 109 160 132 116 90 102 217 148 188 168 264 281 231 100 26 30 75
122 132 100 111 116 156 136 158 80 65 109 148 142 189 195 198 203 180 139 188 128
65 0 91 94 126 145 100 82 60 57 167 99 126 106 208 230 194 36 39 94 103
131 131 101 109 105 129 122 163 39 27 52 81 82 151 184 192 192 199 96 162 184 57
91 0 106 53 115 104 94 74 94 196 134 192 168 251 260 197 126 64 64 19
206 212 179 190 190 224 210 242 129 102 154 186 187 252 273 279 281 269 196 258 2
22 51 94 106 0 158 211 180 163 136 145 259 190 218 200 302 323 278 120 65 49 124
112 106 86 89 81 90 91 135 46 65 22 28 32 104 146 155 153 175 53 119 183 109 126
53 158 0 75 89 88 83 103 178 129 197 173 236 238 166 156 111 115 34
57 44 51 44 34 15 25 65 82 110 63 61 48 44 71 80 78 106 23 48 139 160 145 115 211
```

```
75 0 53 68 86 95 114 90 160 139 173 168 92 162 150 176 101
28 33 4 11 19 59 37 63 65 87 68 92 79 95 95 99 103 95 60 89 95 132 100 104 180 89
53 0 18 44 45 92 42 112 89 149 156 99 111 116 155 97
43 51 18 29 35 75 54 79 55 73 66 97 85 111 113 117 121 109 70 107 95 116 82 94 16
3 88 68 18 0 27 27 103 42 109 85 157 168 115 94 98 140 90
70 77 43 54 57 96 78 106 40 50 61 98 89 130 138 143 147 135 81 129 110 90 60 74 1
36 83 86 44 27 0 21 128 62 119 96 179 192 142 79 72 115 74
65 75 45 56 63 103 81 101 61 68 81 117 106 138 138 141 146 125 95 134 91 102 57 9
4 145 103 95 45 27 21 0 115 46 98 75 163 179 136 67 81 129 95
66 72 95 89 97 105 90 50 157 176 158 173 159 130 81 74 85 21 134 108 62 217 167 1
96 259 178 114 92 103 128 115 0 69 86 81 60 65 54 158 195 243 190
37 52 45 47 58 91 68 66 97 112 107 134 121 127 107 107 115 80 101 114 54 148 99 1
34 190 129 90 42 42 62 46 69 0 71 49 117 133 98 95 127 175 132
103 118 115 118 129 158 136 118 159 166 175 204 191 192 159 155 164 107 172 176 2
4 188 126 192 218 197 160 112 109 119 98 86 71 0 24 94 127 137 100 163 218 194
84 99 93 96 107 139 116 104 135 142 151 181 168 174 146 143 152 100 149 159 23 16
8 106 168 200 173 139 89 85 96 75 81 49 24 0 104 133 127 85 143 197 170
125 132 152 147 156 164 150 109 212 229 216 232 219 186 132 122 133 71 194 163 81
264 208 251 302 236 173 149 157 179 163 60 117 94 104 0 39 100 190 241 292 246
129 132 159 151 158 156 147 103 221 241 219 229 216 172 113 102 112 63 190 147 11
0 281 230 260 323 238 168 156 168 192 179 65 133 127 133 39 0 81 216 259 307 253
72 67 100 90 92 78 76 36 159 184 150 153 140 90 32 22 33 33 115 66 113 231 194 19
7 278 166 92 99 115 142 136 54 98 137 127 100 81 0 193 214 253 187
126 139 112 122 129 169 148 160 110 99 137 176 168 205 200 202 208 173 160 200 10
8 100 36 126 120 156 162 111 94 79 67 158 95 100 85 190 216 193 0 74 129 137
141 148 114 126 127 163 147 178 72 46 100 137 134 193 209 215 218 205 138 197 164
26 39 64 65 111 150 116 98 72 81 195 127 163 143 241 259 214 74 0 55 80
183 186 153 163 161 191 180 218 95 69 115 143 145 214 243 250 251 249 159 224 217
30 94 64 49 115 176 155 140 115 129 243 175 218 197 292 307 253 129 55 0 81
124 122 94 101 95 115 111 153 35 38 37 62 64 135 171 179 178 191 80 147 184 75 10
3 19 124 34 101 97 90 74 95 190 132 194 170 246 253 187 137 80 81 0
EOF
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 2
2, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41]
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0, 8), (0, 9),
(0, 10), (0, 11), (0, 12), (0, 13), (0, 14), (0, 15), (0, 16), (0, 17), (0, 18),
(0, 19), (0, 20), (0, 21), (0, 22), (0, 23), (0, 24), (0, 25), (0, 26), (0, 27),
(0, 28), (0, 29), (0, 30), (0, 31), (0, 32), (0, 33), (0, 34), (0, 35), (0, 36),
(0, 37), (0, 38), (0, 39), (0, 40), (0, 41), (1, 0), (1, 1), (1, 2), (1, 3), (1,
4), (1, 5), (1, 6), (1, 7), (1, 8), (1, 9), (1, 10), (1, 11), (1, 12), (1, 13),
(1, 14), (1, 15), (1, 16), (1, 17), (1, 18), (1, 19), (1, 20), (1, 21), (1, 22),
(1, 23), (1, 24), (1, 25), (1, 26), (1, 27), (1, 28), (1, 29), (1, 30), (1, 31),
(1, 32), (1, 33), (1, 34), (1, 35), (1, 36), (1, 37), (1, 38), (1, 39), (1, 40),
(1, 41), (2, 0), (2, 1), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8),
(2, 9), (2, 10), (2, 11), (2, 12), (2, 13), (2, 14), (2, 15), (2, 16), (2, 17),
(2, 18), (2, 19), (2, 20), (2, 21), (2, 22), (2, 23), (2, 24), (2, 25), (2, 26),
(2, 27), (2, 28), (2, 29), (2, 30), (2, 31), (2, 32), (2, 33), (2, 34), (2, 35),
(2, 36), (2, 37), (2, 38), (2, 39), (2, 40), (2, 41), (3, 0), (3, 1), (3, 2), (3,
3), (3, 4), (3, 5), (3, 6), (3, 7), (3, 8), (3, 9), (3, 10), (3, 11), (3, 12),
(3, 13), (3, 14), (3, 15), (3, 16), (3, 17), (3, 18), (3, 19), (3, 20), (3, 21),
(3, 22), (3, 23), (3, 24), (3, 25), (3, 26), (3, 27), (3, 28), (3, 29), (3, 30),
(3, 31), (3, 32), (3, 33), (3, 34), (3, 35), (3, 36), (3, 37), (3, 38), (3, 39),
(3, 40), (3, 41), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4), (4, 5), (4, 6), (4, 7),
(4, 8), (4, 9), (4, 10), (4, 11), (4, 12), (4, 13), (4, 14), (4, 15), (4, 16),
(4, 17), (4, 18), (4, 19), (4, 20), (4, 21), (4, 22), (4, 23), (4, 24), (4, 25),
(4, 26), (4, 27), (4, 28), (4, 29), (4, 30), (4, 31), (4, 32), (4, 33), (4, 34),
(4, 35), (4, 36), (4, 37), (4, 38), (4, 39), (4, 40), (4, 41), (5, 0), (5, 1),
(5, 2), (5, 3), (5, 4), (5, 5), (5, 6), (5, 7), (5, 8), (5, 9), (5, 10), (5, 11),
(5, 12), (5, 13), (5, 14), (5, 15), (5, 16), (5, 17), (5, 18), (5, 19), (5, 20),
(5, 21), (5, 22), (5, 23), (5, 24), (5, 25), (5, 26), (5, 27), (5, 28), (5, 29),
```

```
(5, 30), (5, 31), (5, 32), (5, 33), (5, 34), (5, 35), (5, 36), (5, 37), (5, 38),
(5, 39), (5, 40), (5, 41), (6, 0), (6, 1), (6, 2), (6, 3), (6, 4), (6, 5), (6,
6), (6, 7), (6, 8), (6, 9), (6, 10), (6, 11), (6, 12), (6, 13), (6, 14), (6, 15),
(6, 16), (6, 17), (6, 18), (6, 19), (6, 20), (6, 21), (6, 22), (6, 23), (6, 24),
(6, 25), (6, 26), (6, 27), (6, 28), (6, 29), (6, 30), (6, 31), (6, 32), (6, 33),
(6, 34), (6, 35), (6, 36), (6, 37), (6, 38), (6, 39), (6, 40), (6, 41), (7, 0),
(7, 1), (7, 2), (7, 3), (7, 4), (7, 5), (7, 6), (7, 7), (7, 8), (7, 9), (7, 10),
(7, 11), (7, 12), (7, 13), (7, 14), (7, 15), (7, 16), (7, 17), (7, 18), (7, 19),
(7, 20), (7, 21), (7, 22), (7, 23), (7, 24), (7, 25), (7, 26), (7, 27), (7, 28),
(7, 29), (7, 30), (7, 31), (7, 32), (7, 33), (7, 34), (7, 35), (7, 36), (7, 37),
(7, 38), (7, 39), (7, 40), (7, 41), (8, 0), (8, 1), (8, 2), (8, 3), (8, 4), (8,
5), (8, 6), (8, 7), (8, 8), (8, 9), (8, 10), (8, 11), (8, 12), (8, 13), (8, 14),
(8, 15), (8, 16), (8, 17), (8, 18), (8, 19), (8, 20), (8, 21), (8, 22), (8, 23),
(8, 24), (8, 25), (8, 26), (8, 27), (8, 28), (8, 29), (8, 30), (8, 31), (8, 32),
(8, 33), (8, 34), (8, 35), (8, 36), (8, 37), (8, 38), (8, 39), (8, 40), (8, 41),
(9, 0), (9, 1), (9, 2), (9, 3), (9, 4), (9, 5), (9, 6), (9, 7), (9, 8), (9, 9),
(9, 10), (9, 11), (9, 12), (9, 13), (9, 14), (9, 15), (9, 16), (9, 17), (9, 18),
(9, 19), (9, 20), (9, 21), (9, 22), (9, 23), (9, 24), (9, 25), (9, 26), (9, 27),
(9, 28), (9, 29), (9, 30), (9, 31), (9, 32), (9, 33), (9, 34), (9, 35), (9, 36),
(9, 37), (9, 38), (9, 39), (9, 40), (9, 41), (10, 0), (10, 1), (10, 2), (10, 3),
(10, 4), (10, 5), (10, 6), (10, 7), (10, 8), (10, 9), (10, 10), (10, 11), (10, 1
2), (10, 13), (10, 14), (10, 15), (10, 16), (10, 17), (10, 18), (10, 19), (10, 2
0), (10, 21), (10, 22), (10, 23), (10, 24), (10, 25), (10, 26), (10, 27), (10, 2
8), (10, 29), (10, 30), (10, 31), (10, 32), (10, 33), (10, 34), (10, 35), (10, 3
6), (10, 37), (10, 38), (10, 39), (10, 40), (10, 41), (11, 0), (11, 1), (11, 2),
(11, 3), (11, 4), (11, 5), (11, 6), (11, 7), (11, 8), (11, 9), (11, 10), (11, 1
1), (11, 12), (11, 13), (11, 14), (11, 15), (11, 16), (11, 17), (11, 18), (11, 1
9), (11, 20), (11, 21), (11, 22), (11, 23), (11, 24), (11, 25), (11, 26), (11, 2
7), (11, 28), (11, 29), (11, 30), (11, 31), (11, 32), (11, 33), (11, 34), (11, 3
5), (11, 36), (11, 37), (11, 38), (11, 39), (11, 40), (11, 41), (12, 0), (12, 1),
(12, 2), (12, 3), (12, 4), (12, 5), (12, 6), (12, 7), (12, 8), (12, 9), (12, 10),
(12, 11), (12, 12), (12, 13), (12, 14), (12, 15), (12, 16), (12, 17), (12, 18),
(12, 19), (12, 20), (12, 21), (12, 22), (12, 23), (12, 24), (12, 25), (12, 26),
(12, 27), (12, 28), (12, 29), (12, 30), (12, 31), (12, 32), (12, 33), (12, 34),
(12, 35), (12, 36), (12, 37), (12, 38), (12, 39), (12, 40), (12, 41), (13, 0), (1
3, 1), (13, 2), (13, 3), (13, 4), (13, 5), (13, 6), (13, 7), (13, 8), (13, 9), (1
3, 10), (13, 11), (13, 12), (13, 13), (13, 14), (13, 15), (13, 16), (13, 17), (1
3, 18), (13, 19), (13, 20), (13, 21), (13, 22), (13, 23), (13, 24), (13, 25), (1
3, 26), (13, 27), (13, 28), (13, 29), (13, 30), (13, 31), (13, 32), (13, 33), (1
3, 34), (13, 35), (13, 36), (13, 37), (13, 38), (13, 39), (13, 40), (13, 41), (1
4, 0), (14, 1), (14, 2), (14, 3), (14, 4), (14, 5), (14, 6), (14, 7), (14, 8), (1
4, 9), (14, 10), (14, 11), (14, 12), (14, 13), (14, 14), (14, 15), (14, 16), (14,
17), (14, 18), (14, 19), (14, 20), (14, 21), (14, 22), (14, 23), (14, 24), (14, 2
5), (14, 26), (14, 27), (14, 28), (14, 29), (14, 30), (14, 31), (14, 32), (14, 3
3), (14, 34), (14, 35), (14, 36), (14, 37), (14, 38), (14, 39), (14, 40), (14, 4
1), (15, 0), (15, 1), (15, 2), (15, 3), (15, 4), (15, 5), (15, 6), (15, 7), (15,
8), (15, 9), (15, 10), (15, 11), (15, 12), (15, 13), (15, 14), (15, 15), (15, 1
6), (15, 17), (15, 18), (15, 19), (15, 20), (15, 21), (15, 22), (15, 23), (15, 2
4), (15, 25), (15, 26), (15, 27), (15, 28), (15, 29), (15, 30), (15, 31), (15, 3
2), (15, 33), (15, 34), (15, 35), (15, 36), (15, 37), (15, 38), (15, 39), (15, 4
0), (15, 41), (16, 0), (16, 1), (16, 2), (16, 3), (16, 4), (16, 5), (16, 6), (16,
7), (16, 8), (16, 9), (16, 10), (16, 11), (16, 12), (16, 13), (16, 14), (16, 15),
(16, 16), (16, 17), (16, 18), (16, 19), (16, 20), (16, 21), (16, 22), (16, 23),
(16, 24), (16, 25), (16, 26), (16, 27), (16, 28), (16, 29), (16, 30), (16, 31),
(16, 32), (16, 33), (16, 34), (16, 35), (16, 36), (16, 37), (16, 38), (16, 39),
(16, 40), (16, 41), (17, 0), (17, 1), (17, 2), (17, 3), (17, 4), (17, 5), (17,
6), (17, 7), (17, 8), (17, 9), (17, 10), (17, 11), (17, 12), (17, 13), (17, 14),
(17, 15), (17, 16), (17, 17), (17, 18), (17, 19), (17, 20), (17, 21), (17, 22),
(17, 23), (17, 24), (17, 25), (17, 26), (17, 27), (17, 28), (17, 29), (17, 30),
(17, 31), (17, 32), (17, 33), (17, 34), (17, 35), (17, 36), (17, 37), (17, 38),
```

```
(17, 39), (17, 40), (17, 41), (18, 0), (18, 1), (18, 2), (18, 3), (18, 4), (18,
5), (18, 6), (18, 7), (18, 8), (18, 9), (18, 10), (18, 11), (18, 12), (18, 13),
(18, 14), (18, 15), (18, 16), (18, 17), (18, 18), (18, 19), (18, 20), (18, 21),
(18, 22), (18, 23), (18, 24), (18, 25), (18, 26), (18, 27), (18, 28), (18, 29),
(18, 30), (18, 31), (18, 32), (18, 33), (18, 34), (18, 35), (18, 36), (18, 37),
(18, 38), (18, 39), (18, 40), (18, 41), (19, 0), (19, 1), (19, 2), (19, 3), (19,
4), (19, 5), (19, 6), (19, 7), (19, 8), (19, 9), (19, 10), (19, 11), (19, 12), (1
9, 13), (19, 14), (19, 15), (19, 16), (19, 17), (19, 18), (19, 19), (19, 20), (1
9, 21), (19, 22), (19, 23), (19, 24), (19, 25), (19, 26), (19, 27), (19, 28), (1
9, 29), (19, 30), (19, 31), (19, 32), (19, 33), (19, 34), (19, 35), (19, 36), (1
9, 37), (19, 38), (19, 39), (19, 40), (19, 41), (20, 0), (20, 1), (20, 2), (20,
3), (20, 4), (20, 5), (20, 6), (20, 7), (20, 8), (20, 9), (20, 10), (20, 11), (2
0, 12), (20, 13), (20, 14), (20, 15), (20, 16), (20, 17), (20, 18), (20, 19), (2
0, 20), (20, 21), (20, 22), (20, 23), (20, 24), (20, 25), (20, 26), (20, 27), (2
0, 28), (20, 29), (20, 30), (20, 31), (20, 32), (20, 33), (20, 34), (20, 35), (2
0, 36), (20, 37), (20, 38), (20, 39), (20, 40), (20, 41), (21, 0), (21, 1), (21,
2), (21, 3), (21, 4), (21, 5), (21, 6), (21, 7), (21, 8), (21, 9), (21, 10), (21,
11), (21, 12), (21, 13), (21, 14), (21, 15), (21, 16), (21, 17), (21, 18), (21, 1
9), (21, 20), (21, 21), (21, 22), (21, 23), (21, 24), (21, 25), (21, 26), (21, 2
7), (21, 28), (21, 29), (21, 30), (21, 31), (21, 32), (21, 33), (21, 34), (21, 3
5), (21, 36), (21, 37), (21, 38), (21, 39), (21, 40), (21, 41), (22, 0), (22, 1),
(22, 2), (22, 3), (22, 4), (22, 5), (22, 6), (22, 7), (22, 8), (22, 9), (22, 10),
(22, 11), (22, 12), (22, 13), (22, 14), (22, 15), (22, 16), (22, 17), (22, 18),
(22, 19), (22, 20), (22, 21), (22, 22), (22, 23), (22, 24), (22, 25), (22, 26),
(22, 27), (22, 28), (22, 29), (22, 30), (22, 31), (22, 32), (22, 33), (22, 34),
(22, 35), (22, 36), (22, 37), (22, 38), (22, 39), (22, 40), (22, 41), (23, 0), (2
3, 1), (23, 2), (23, 3), (23, 4), (23, 5), (23, 6), (23, 7), (23, 8), (23, 9), (2
3, 10), (23, 11), (23, 12), (23, 13), (23, 14), (23, 15), (23, 16), (23, 17), (2
3, 18), (23, 19), (23, 20), (23, 21), (23, 22), (23, 23), (23, 24), (23, 25), (2
3, 26), (23, 27), (23, 28), (23, 29), (23, 30), (23, 31), (23, 32), (23, 33), (2
3, 34), (23, 35), (23, 36), (23, 37), (23, 38), (23, 39), (23, 40), (23, 41), (2
4, 0), (24, 1), (24, 2), (24, 3), (24, 4), (24, 5), (24, 6), (24, 7), (24, 8), (2
4, 9), (24, 10), (24, 11), (24, 12), (24, 13), (24, 14), (24, 15), (24, 16), (24,
17), (24, 18), (24, 19), (24, 20), (24, 21), (24, 22), (24, 23), (24, 24), (24, 2
5), (24, 26), (24, 27), (24, 28), (24, 29), (24, 30), (24, 31), (24, 32), (24, 3
3), (24, 34), (24, 35), (24, 36), (24, 37), (24, 38), (24, 39), (24, 40), (24, 4
1), (25, 0), (25, 1), (25, 2), (25, 3), (25, 4), (25, 5), (25, 6), (25, 7), (25,
8), (25, 9), (25, 10), (25, 11), (25, 12), (25, 13), (25, 14), (25, 15), (25, 1
6), (25, 17), (25, 18), (25, 19), (25, 20), (25, 21), (25, 22), (25, 23), (25, 2
4), (25, 25), (25, 26), (25, 27), (25, 28), (25, 29), (25, 30), (25, 31), (25, 3
2), (25, 33), (25, 34), (25, 35), (25, 36), (25, 37), (25, 38), (25, 39), (25, 4
0), (25, 41), (26, 0), (26, 1), (26, 2), (26, 3), (26, 4), (26, 5), (26, 6), (26,
7), (26, 8), (26, 9), (26, 10), (26, 11), (26, 12), (26, 13), (26, 14), (26, 15),
(26, 16), (26, 17), (26, 18), (26, 19), (26, 20), (26, 21), (26, 22), (26, 23),
(26, 24), (26, 25), (26, 26), (26, 27), (26, 28), (26, 29), (26, 30), (26, 31),
(26, 32), (26, 33), (26, 34), (26, 35), (26, 36), (26, 37), (26, 38), (26, 39),
(26, 40), (26, 41), (27, 0), (27, 1), (27, 2), (27, 3), (27, 4), (27, 5), (27,
6), (27, 7), (27, 8), (27, 9), (27, 10), (27, 11), (27, 12), (27, 13), (27, 14),
(27, 15), (27, 16), (27, 17), (27, 18), (27, 19), (27, 20), (27, 21), (27, 22),
(27, 23), (27, 24), (27, 25), (27, 26), (27, 27), (27, 28), (27, 29), (27, 30),
(27, 31), (27, 32), (27, 33), (27, 34), (27, 35), (27, 36), (27, 37), (27, 38),
(27, 39), (27, 40), (27, 41), (28, 0), (28, 1), (28, 2), (28, 3), (28, 4), (28,
5), (28, 6), (28, 7), (28, 8), (28, 9), (28, 10), (28, 11), (28, 12), (28, 13),
(28, 14), (28, 15), (28, 16), (28, 17), (28, 18), (28, 19), (28, 20), (28, 21),
(28, 22), (28, 23), (28, 24), (28, 25), (28, 26), (28, 27), (28, 28), (28, 29),
(28, 30), (28, 31), (28, 32), (28, 33), (28, 34), (28, 35), (28, 36), (28, 37),
(28, 38), (28, 39), (28, 40), (28, 41), (29, 0), (29, 1), (29, 2), (29, 3), (29,
4), (29, 5), (29, 6), (29, 7), (29, 8), (29, 9), (29, 10), (29, 11), (29, 12), (2
9, 13), (29, 14), (29, 15), (29, 16), (29, 17), (29, 18), (29, 19), (29, 20), (2
9, 21), (29, 22), (29, 23), (29, 24), (29, 25), (29, 26), (29, 27), (29, 28), (2
```

```
9, 29), (29, 30), (29, 31), (29, 32), (29, 33), (29, 34), (29, 35), (29, 36), (2
9, 37), (29, 38), (29, 39), (29, 40), (29, 41), (30, 0), (30, 1), (30, 2), (30,
3), (30, 4), (30, 5), (30, 6), (30, 7), (30, 8), (30, 9), (30, 10), (30, 11), (3
0, 12), (30, 13), (30, 14), (30, 15), (30, 16), (30, 17), (30, 18), (30, 19), (3
0, 20), (30, 21), (30, 22), (30, 23), (30, 24), (30, 25), (30, 26), (30, 27), (3
0, 28), (30, 29), (30, 30), (30, 31), (30, 32), (30, 33), (30, 34), (30, 35), (3
0, 36), (30, 37), (30, 38), (30, 39), (30, 40), (30, 41), (31, 0), (31, 1), (31,
2), (31, 3), (31, 4), (31, 5), (31, 6), (31, 7), (31, 8), (31, 9), (31, 10), (31,
11), (31, 12), (31, 13), (31, 14), (31, 15), (31, 16), (31, 17), (31, 18), (31, 1
9), (31, 20), (31, 21), (31, 22), (31, 23), (31, 24), (31, 25), (31, 26), (31, 2
7), (31, 28), (31, 29), (31, 30), (31, 31), (31, 32), (31, 33), (31, 34), (31, 3
5), (31, 36), (31, 37), (31, 38), (31, 39), (31, 40), (31, 41), (32, 0), (32, 1),
(32, 2), (32, 3), (32, 4), (32, 5), (32, 6), (32, 7), (32, 8), (32, 9), (32, 10),
(32, 11), (32, 12), (32, 13), (32, 14), (32, 15), (32, 16), (32, 17), (32, 18),
(32, 19), (32, 20), (32, 21), (32, 22), (32, 23), (32, 24), (32, 25), (32, 26),
(32, 27), (32, 28), (32, 29), (32, 30), (32, 31), (32, 32), (32, 33), (32, 34),
(32, 35), (32, 36), (32, 37), (32, 38), (32, 39), (32, 40), (32, 41), (33, 0), (3
3, 1), (33, 2), (33, 3), (33, 4), (33, 5), (33, 6), (33, 7), (33, 8), (33, 9), (3
3, 10), (33, 11), (33, 12), (33, 13), (33, 14), (33, 15), (33, 16), (33, 17), (3
3, 18), (33, 19), (33, 20), (33, 21), (33, 22), (33, 23), (33, 24), (33, 25), (3
3, 26), (33, 27), (33, 28), (33, 29), (33, 30), (33, 31), (33, 32), (33, 33), (3
3, 34), (33, 35), (33, 36), (33, 37), (33, 38), (33, 39), (33, 40), (33, 41), (3
4, 0), (34, 1), (34, 2), (34, 3), (34, 4), (34, 5), (34, 6), (34, 7), (34, 8), (3
4, 9), (34, 10), (34, 11), (34, 12), (34, 13), (34, 14), (34, 15), (34, 16), (34,
17), (34, 18), (34, 19), (34, 20), (34, 21), (34, 22), (34, 23), (34, 24), (34, 2
5), (34, 26), (34, 27), (34, 28), (34, 29), (34, 30), (34, 31), (34, 32), (34, 3
3), (34, 34), (34, 35), (34, 36), (34, 37), (34, 38), (34, 39), (34, 40), (34, 4
1), (35, 0), (35, 1), (35, 2), (35, 3), (35, 4), (35, 5), (35, 6), (35, 7), (35,
8), (35, 9), (35, 10), (35, 11), (35, 12), (35, 13), (35, 14), (35, 15), (35, 1
6), (35, 17), (35, 18), (35, 19), (35, 20), (35, 21), (35, 22), (35, 23), (35, 2
4), (35, 25), (35, 26), (35, 27), (35, 28), (35, 29), (35, 30), (35, 31), (35, 3
2), (35, 33), (35, 34), (35, 35), (35, 36), (35, 37), (35, 38), (35, 39), (35, 4
0), (35, 41), (36, 0), (36, 1), (36, 2), (36, 3), (36, 4), (36, 5), (36, 6), (36,
7), (36, 8), (36, 9), (36, 10), (36, 11), (36, 12), (36, 13), (36, 14), (36, 15),
(36, 16), (36, 17), (36, 18), (36, 19), (36, 20), (36, 21), (36, 22), (36, 23),
(36, 24), (36, 25), (36, 26), (36, 27), (36, 28), (36, 29), (36, 30), (36, 31),
(36, 32), (36, 33), (36, 34), (36, 35), (36, 36), (36, 37), (36, 38), (36, 39),
(36, 40), (36, 41), (37, 0), (37, 1), (37, 2), (37, 3), (37, 4), (37, 5), (37,
6), (37, 7), (37, 8), (37, 9), (37, 10), (37, 11), (37, 12), (37, 13), (37, 14),
(37, 15), (37, 16), (37, 17), (37, 18), (37, 19), (37, 20), (37, 21), (37, 22),
(37, 23), (37, 24), (37, 25), (37, 26), (37, 27), (37, 28), (37, 29), (37, 30),
(37, 31), (37, 32), (37, 33), (37, 34), (37, 35), (37, 36), (37, 37), (37, 38),
(37, 39), (37, 40), (37, 41), (38, 0), (38, 1), (38, 2), (38, 3), (38, 4), (38,
5), (38, 6), (38, 7), (38, 8), (38, 9), (38, 10), (38, 11), (38, 12), (38, 13),
(38, 14), (38, 15), (38, 16), (38, 17), (38, 18), (38, 19), (38, 20), (38, 21),
(38, 22), (38, 23), (38, 24), (38, 25), (38, 26), (38, 27), (38, 28), (38, 29),
(38, 30), (38, 31), (38, 32), (38, 33), (38, 34), (38, 35), (38, 36), (38, 37),
(38, 38), (38, 39), (38, 40), (38, 41), (39, 0), (39, 1), (39, 2), (39, 3), (39,
4), (39, 5), (39, 6), (39, 7), (39, 8), (39, 9), (39, 10), (39, 11), (39, 12), (3
9, 13), (39, 14), (39, 15), (39, 16), (39, 17), (39, 18), (39, 19), (39, 20), (3
9, 21), (39, 22), (39, 23), (39, 24), (39, 25), (39, 26), (39, 27), (39, 28), (3
9, 29), (39, 30), (39, 31), (39, 32), (39, 33), (39, 34), (39, 35), (39, 36), (3
9, 37), (39, 38), (39, 39), (39, 40), (39, 41), (40, 0), (40, 1), (40, 2), (40,
3), (40, 4), (40, 5), (40, 6), (40, 7), (40, 8), (40, 9), (40, 10), (40, 11), (4
0, 12), (40, 13), (40, 14), (40, 15), (40, 16), (40, 17), (40, 18), (40, 19), (4
0, 20), (40, 21), (40, 22), (40, 23), (40, 24), (40, 25), (40, 26), (40, 27), (4
0, 28), (40, 29), (40, 30), (40, 31), (40, 32), (40, 33), (40, 34), (40, 35), (4
0, 36), (40, 37), (40, 38), (40, 39), (40, 40), (40, 41), (41, 0), (41, 1), (41,
2), (41, 3), (41, 4), (41, 5), (41, 6), (41, 7), (41, 8), (41, 9), (41, 10), (41,
11), (41, 12), (41, 13), (41, 14), (41, 15), (41, 16), (41, 17), (41, 18), (41, 1
```

```
9), (41, 20), (41, 21), (41, 22), (41, 23), (41, 24), (41, 25), (41, 26), (41, 2
7), (41, 28), (41, 29), (41, 30), (41, 31), (41, 32), (41, 33), (41, 34), (41, 3
5), (41, 36), (41, 37), (41, 38), (41, 39), (41, 40), (41, 41)]
```

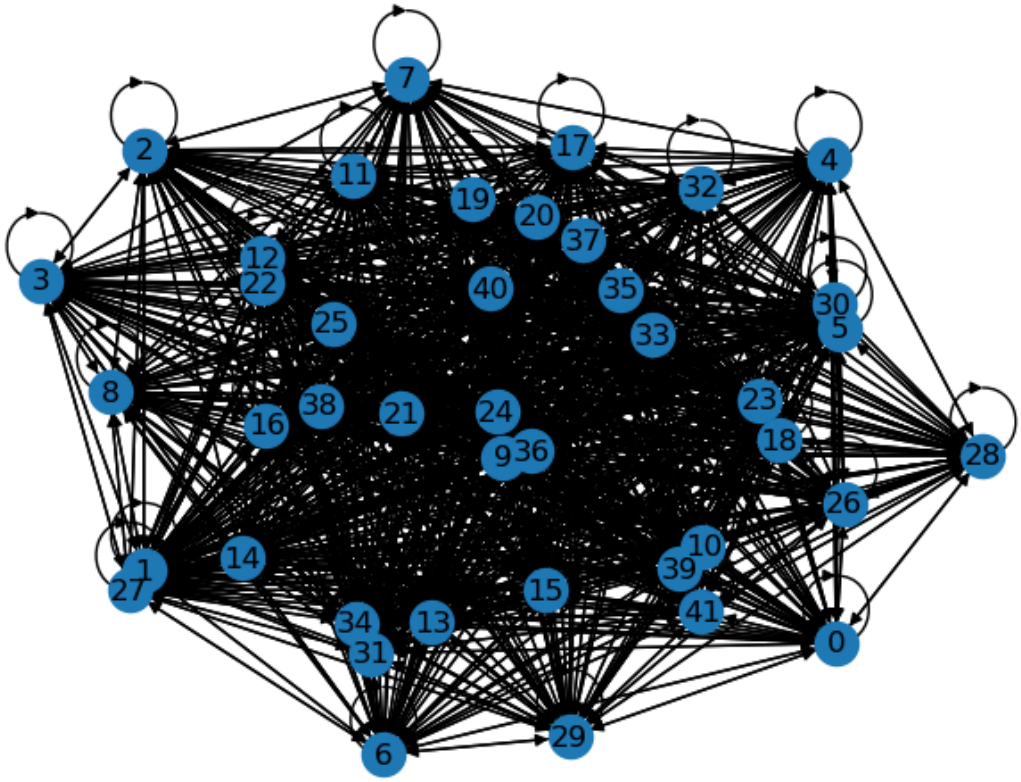
In [4]:
```python
# Probamos algunas funciones del objeto problema, que ya nos vienen dadas

# Distancia entre nodos (ciudades)
distancia = problema.get_weight(3, 0)
print(distancia)

#Todas las funciones
#Documentación: https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
#dir(problema)

import networkx as nx
import matplotlib.pyplot as plt
nx.draw(problema.get_graph(), with_labels=True, arrows=True)
plt.show()
```

```
23
```



In [5]:
```python
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.manifold import MDS  # Multidimensional Scaling o Escalado Multidim

def plot_tsp_solution(distance_matrix, tsp_solution):
    """
    Dibuja el grafo de un TSP con las posiciones calculadas mediante MDS y muest
    solo las aristas correspondientes a la solución del TSP.

    :param distance_matrix: np.ndarray, matriz de distancias entre nodos
    :param tsp_solution: list, lista de nodos en el orden de la solución del TSP
    """
```

```
    # Crear el grafo completo
    G = nx.Graph()
    num_nodes = len(distance_matrix)
    for i in range(num_nodes):
        for j in range(i + 1, num_nodes):
            G.add_edge(i, j, weight=distance_matrix[i][j])

    # Usar MDS para calcular posiciones de los nodos
    mds = MDS(n_components=2, dissimilarity="precomputed", random_state=42)
    positions = mds.fit_transform(distance_matrix)

    # Convertir las posiciones en un diccionario para networkx
    pos = {i: positions[i] for i in range(num_nodes)}

    # Crear un subgrafo con las aristas del camino TSP
    TSP_G = nx.Graph()
    for i in range(len(tsp_solution) - 1):
        u = tsp_solution[i]
        v = tsp_solution[i + 1]
        TSP_G.add_edge(u, v, weight=distance_matrix[u][v])

    # Dibujar el grafo
    plt.figure(figsize=(8, 6))

    # Dibujar nodos
    nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=500)

    # Dibujar las aristas del camino TSP
    nx.draw_networkx_edges(TSP_G, pos, edge_color='red', width=2)

    # Añadir etiquetas a los nodos y pesos de las aristas
    nx.draw_networkx_labels(G, pos, font_size=10, font_weight='bold')
    edge_labels = nx.get_edge_attributes(TSP_G, 'weight')
    nx.draw_networkx_edge_labels(TSP_G, pos, edge_labels=edge_labels, font_size=

    plt.title("Grafo TSP con solución específica")
    plt.show()
```

Funciones básicas para algoritmos de búsqueda

In [6]:

```
# Genera una solución aleatoria con comienzo en en el nodo 0
def crear_solucion(nodos):
    # Nodo inicial = 0
    solucion = [nodos[0]]
    # Para el resto de nodos, los recorremos e incluimos aleatoriamente en la so
    for n in nodos[1:]:
        # Debemos quitar el nodo 0 y los ya incluidos en cada iteración
        solucion = solucion + [random.choice(list(set(nodos) - set({nodos[0]}) -
    return solucion


# Devuelve la distancia entre dos nodos aprovechando la función get_weight() de
def distancia(a,b, problema):
    return problema.get_weight(a,b)


# Devuelve la distancia total de una trayectoria/solución
def distancia_total(solucion, problema):
```

```
    distancia_total = 0
    # Para cada nodo de la solución calculamos su distancia a siguiente nodo
    for i in range(len(solucion)-1):
        distancia_total += distancia(solucion[i] ,solucion[i+1] ,  problema)
    # Devolvemos la distancia total más la distancia del último nodo al primero
    return distancia_total + distancia(solucion[len(solucion)-1] ,solucion[0], p


solucion = crear_solucion(nodos)
print(solucion)
print(distancia_total(solucion, problema))
```

```
[0, 21, 27, 38, 13, 2, 36, 16, 34, 25, 15, 11, 32, 30, 3, 31, 20, 10, 6, 41, 4, 3
5, 7, 22, 8, 40, 23, 9, 5, 29, 28, 24, 14, 19, 37, 39, 12, 17, 33, 26, 18, 1]
4759
```

Búsqueda aleatoria - TSP

- El algoritmo inicializa soluciones al azar y se queda con la mejor que encuentre
- Se realizan N iteraciones y nos quedamos con la mejor
- Todo es azar, sin que garanticemos solución óptima

In [7]:
```
def busqueda_aleatoria(problema, iteraciones):
    # Obtenemos listado de nodos del problema
    nodos = list(problema.get_nodes())

    mejor_solucion = []
    mejor_distancia_total = float("inf")

    for i in range(iteraciones):
        # Generamos solución aleatoria
        solucion = crear_solucion(nodos)
        # Calculamos distancia total de la solución
        distancia = distancia_total(solucion, problema)
        if (distancia < mejor_distancia_total):
            mejor_solucion = solucion
            mejor_distancia_total = distancia

    return mejor_solucion, mejor_distancia_total

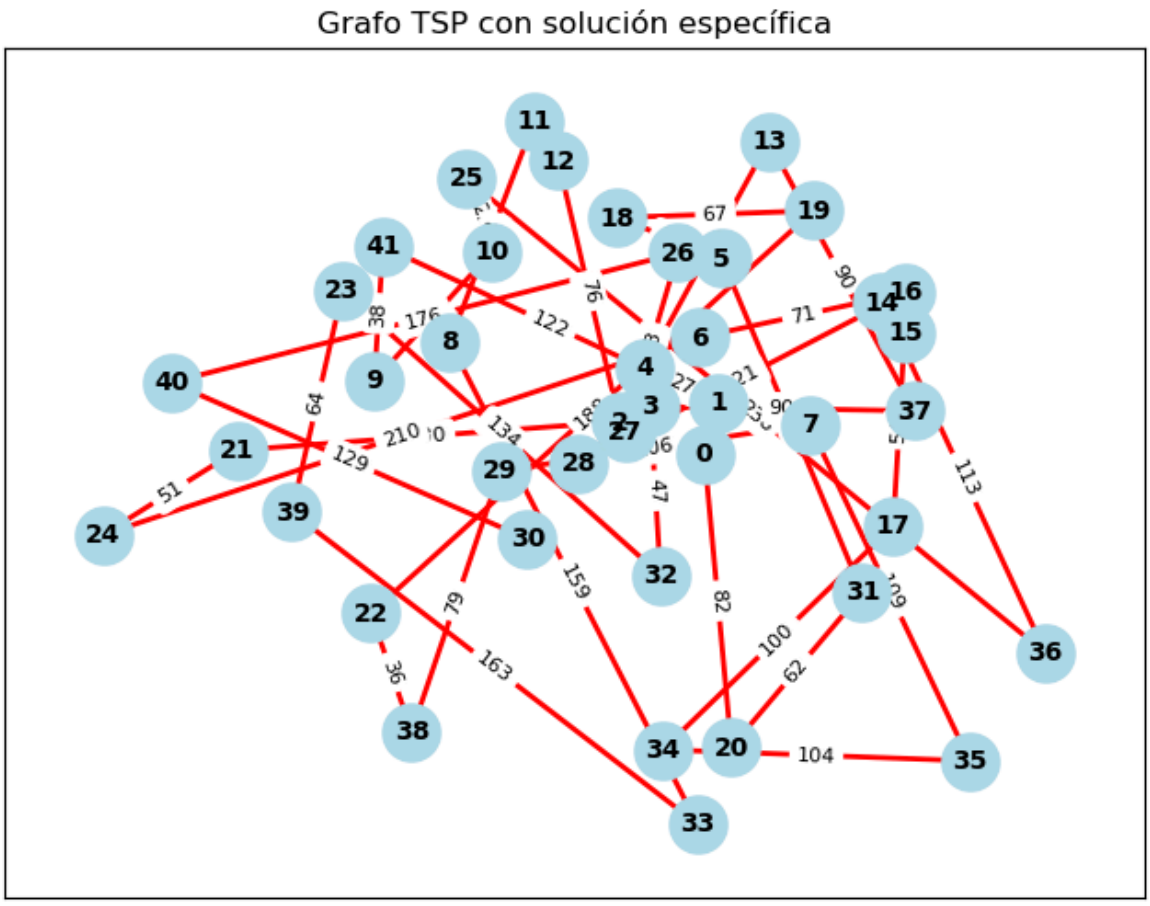
iteraciones = 5000
mejor_solucion, mejor_distancia_total = busqueda_aleatoria(problema, iteraciones

print(f"Solución: {mejor_solucion}")
print(f"Distancia: {mejor_distancia_total}")

plot_tsp_solution(problema.edge_weights, mejor_solucion)
```

```
Solución: [0, 20, 31, 5, 18, 19, 22, 38, 29, 7, 35, 34, 17, 15, 14, 36, 25, 10,
9, 41, 1, 4, 13, 37, 3, 32, 23, 39, 33, 8, 11, 12, 2, 21, 24, 6, 16, 28, 27, 26,
40, 30]
Distancia: 3759
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\manifold\_mds.py:677: FutureWa
rning: The default value of `n_init` will change from 4 to 1 in 1.9.
  warnings.warn(
```



Búsqueda local - TSP

- El algoritmo se va quedando con vecinos (operadores de vecindad) para intentar converger a mejores soluciones
- Se realizan N iteraciones y nos quedamos con la mejor

In [8]:
```
# Función que a partir de una solución genera la vecina óptima (es decir, la mej
def genera_mejor_vecina(solucion):
    # Generador de soluciones vecinas: 2-opt (intercambiar 2 nodos) Si hay N nod
    # Se puede modificar para aplicar otros generadores distintos que 2-opt como
    #print(solucion)

    mejor_solucion = []
    mejor_distancia = float("inf")

    #Recorremos todos los nodos en bucle doble para evaluar todos los intercambi
    for i in range(1,len(solucion)-1):
        for j in range(i+1, len(solucion)):
            # Generamos nueva solución vecina (0, 1, 2, 3, i, ..., j, ..., 41) -
            vecina = solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[

            # Se evalua la nueva solución
            distancia_vecina = distancia_total(vecina, problema)

            # Si es la mejor solución, la almacenamos
            if (distancia_vecina <= mejor_distancia):
                mejor_distancia = distancia_vecina
                mejor_solucion = vecina
```

```
    # Devolvemos mejor vecina
    return mejor_solucion


solucion = crear_solucion(nodos)
print(f"Distancia Solucion Incial: {distancia_total(solucion, problema)}")

nueva_solucion = genera_mejor_vecina(solucion)
print(f"Distancia Mejor Solucion Local: {distancia_total(nueva_solucion, problem
```

```
Distancia Solucion Incial: 5046
Distancia Mejor Solucion Local: 4279
```

In [9]:

```
# En este caso, la búsqueda se ejecuta sin criterio de parada, se para cuando no
# Aquí iremos recorriendo las mejores vecinas para ir construyendo trayectoria
def busqueda_local(problema):
    mejor_solucion = []

    # Generar una solucion inicial de referencia(aleatoria)
    solucion_referencia = crear_solucion(nodos)
    mejor_distancia = distancia_total(solucion_referencia, problema)

    # Contador de iteraciones (por tener referencia)
    iteracion = 0

    while(1):
        iteracion += 1
        #print('#',iteracion)

        # Obtenemos la mejor vecina
        vecina = genera_mejor_vecina(solucion_referencia)

        # Evaluamos para ver si mejoramos respecto a lo encontrado hasta el mome
        distancia_vecina = distancia_total(vecina, problema)

        # Si mejoramos hay que seguir
        if (distancia_vecina < mejor_distancia):
            #mejor_solucion = copy.deepcopy(vecina)   # Con copia profunda. Las
            mejor_solucion = vecina                   # Guarda la mejor solución
            mejor_distancia = distancia_vecina

        # Si no mejoramos, retornamos ya que hemos llegado a un minimo local (se
        else:
            #print(f"En la iteracion {iteracion} la mejor solución encontrada es
            #print(f"Distancia: {mejor_distancia}")
            return mejor_solucion, mejor_distancia, iteracion

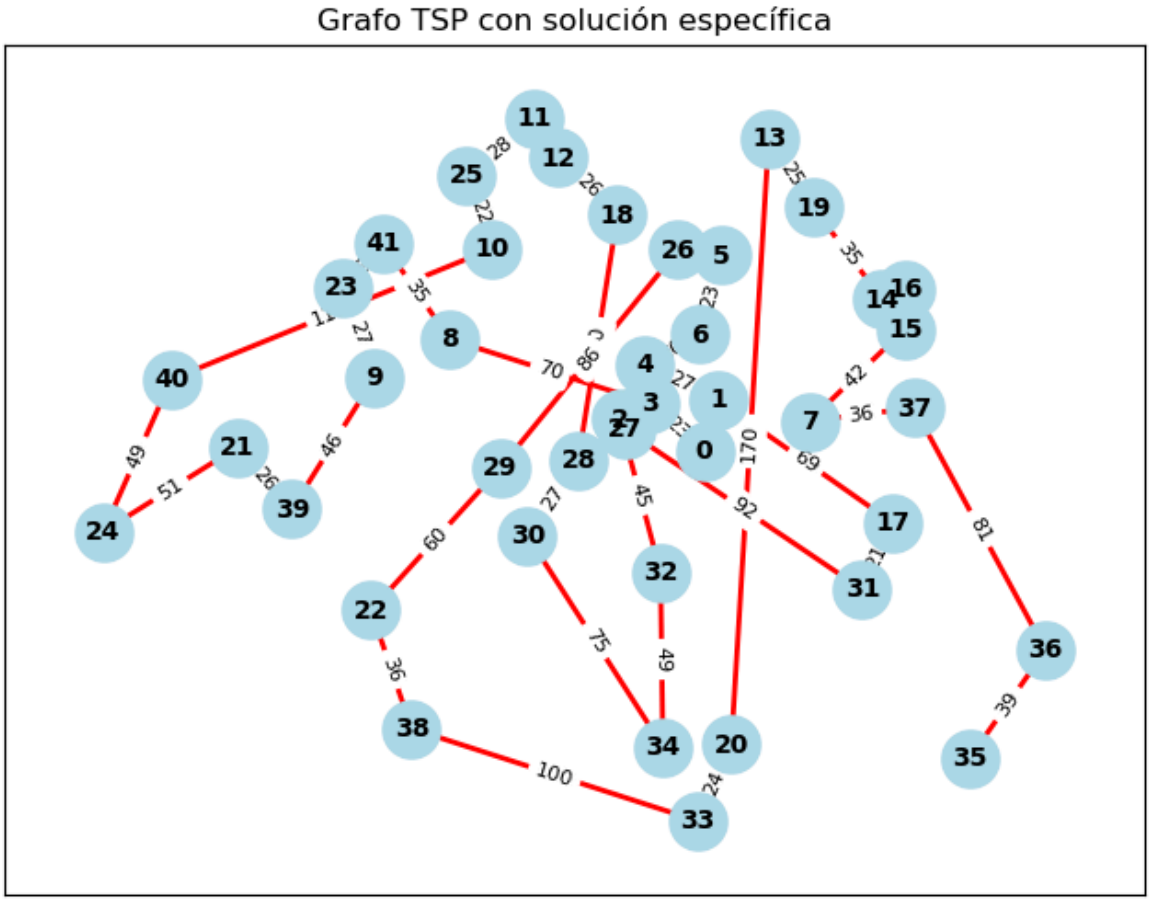
        solucion_referencia = vecina


mejor_solucion, mejor_distancia, iteracion = busqueda_local(problema)
print(f"En la iteracion {iteracion} la mejor solución encontrada es: {mejor_solu
print(f"Distancia: {mejor_distancia}")

plot_tsp_solution(problema.edge_weights, mejor_solucion)
```

```
En la iteracion 30 la mejor solución encontrada es: [0, 3, 8, 41, 23, 9, 39, 21,
24, 40, 10, 25, 11, 12, 18, 28, 30, 34, 32, 2, 27, 31, 17, 1, 4, 6, 5, 26, 29, 2
2, 38, 33, 20, 13, 19, 14, 16, 15, 7, 37, 36, 35]
Distancia: 1966
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\manifold\_mds.py:677: FutureWa
rning: The default value of `n_init` will change from 4 to 1 in 1.9.
  warnings.warn(
```



Recocido simulado (Simulated Annealing) - TSP

- El algoritmo se va quedando con vecinos (operadores de vecindad) para intentar converger a mejores soluciones, aunque ahora a veces puedo elegir vecinos peores
- Al principio, elijo más veces vecinos peores (diversifico) y luego elijo menos veces esos vecinos peores (intensifico)
- Se realizan N iteraciones o bajamos temperatura al máximo

Funciones auxiliares

In [10]:
```
# Generador de 1 solucion vecina 2-opt 100% aleatoria (intercambiar 2 nodos)
def genera_vecina_aleatorio(solucion):

  # Se eligen dos nodos aleatoriamente
  i,j = sorted(random.sample(range(1,len(solucion)) , 2))

  # Devuelve una nueva solución pero intercambiando los dos nodos elegidos al az
  return solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + soluci


# Funcion de probabilidad para aceptar peores soluciones
def probabilidad(T, d):
  if random.random() <  math.exp(-1*d / T)  :
    return True
  else:
```

```
    return False


# Funcion de descenso de temperatura
def bajar_temperatura(T):
  return T*0.9999
```

In [11]:
```
def recocido_simulado(problema, temperatura):
    # Generamos solución de referencia
    solucion_referencia = crear_solucion(nodos)
    distancia_referencia = distancia_total(solucion_referencia, problema)

    mejor_solucion = []
    mejor_distancia = float("inf")

    iteraciones = 0

    # Mientras la temperatura no sea 0 (aquí no hay parada por iteraciones)
    while temperatura > .0001:
        iteraciones += 1

        # Genera una solución vecina
        vecina = genera_vecina_aleatorio(solucion_referencia)

        # Calculamos distancia
        distancia_vecina = distancia_total(vecina, problema)

        # Si es la mejor solución de todas se guarda siempre
        if (distancia_vecina < mejor_distancia):
            mejor_solucion = vecina
            mejor_distancia = distancia_vecina

        # Si la nueva vecina es o
        # Si el vecino es peor se cambia según una probabilidad que depende de T
        if ((distancia_vecina < distancia_referencia) or probabilidad(temperatur
            #solucion_referencia = copy.deepcopy(vecina)
            solucion_referencia = vecina
            distancia_referencia = distancia_vecina

        # Bajamos la temperatura
        temperatura = bajar_temperatura(temperatura)

    return mejor_solucion, mejor_distancia, iteraciones

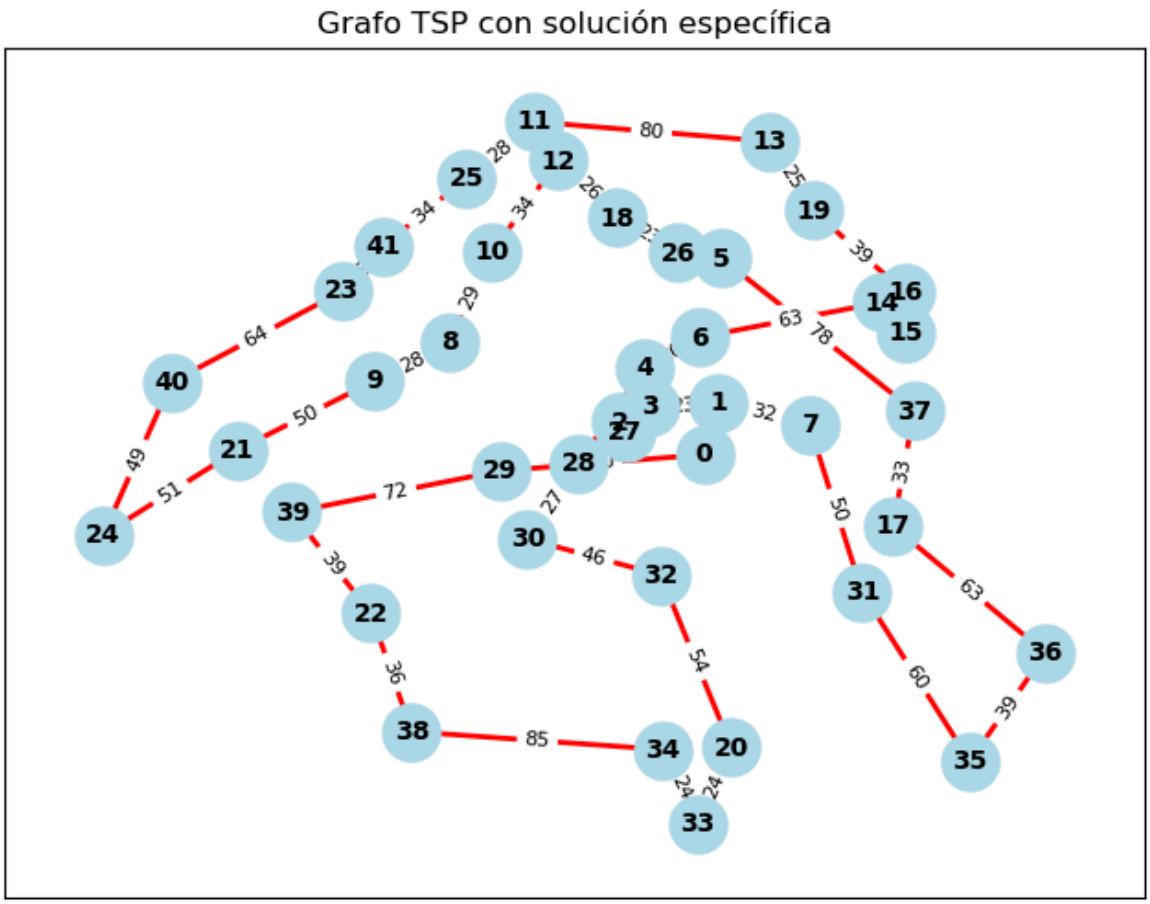
temperatura = 100000
mejor_solucion, mejor_distancia, iteraciones  = recocido_simulado(problema, temp

print(f"La mejor solución encontrada es: {mejor_solucion} en {iteraciones} itera
print(f"Distancia: {mejor_distancia}")

plot_tsp_solution(problema.edge_weights, mejor_solucion)
```

```
La mejor solución encontrada es: [0, 29, 39, 22, 38, 34, 33, 20, 32, 30, 28, 4,
6, 14, 15, 16, 19, 13, 11, 25, 41, 23, 40, 24, 21, 9, 8, 10, 12, 18, 26, 5, 37, 1
7, 36, 35, 31, 7, 1, 3, 2, 27] en 207223 iteraciones
Distancia: 1632
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\manifold\_mds.py:677: FutureWa
rning: The default value of `n_init` will change from 4 to 1 in 1.9.
  warnings.warn(
```



Actividades extra

- Mejora de algoritmo de búsqueda local
  - Multiarranque
  - Búsqueda en entornos variables
- Recocido simulado
  - Mejora en la selección de vecinos

Mejora de algoritmo de búsqueda local

La búsqueda local del bloque anterior depende por completo de dónde arranca: un solo arranque aleatorio puede quedar atrapado en un mínimo local mediocre, y usa un único operador de vecindad (2-opt), que no puede alcanzar todos los movimientos posibles. Resuelvo aquí las dos mejoras que pide la guía, cada una atacando una de esas dos limitaciones por separado.

Multiarranque

Repito `busqueda_local()` completa desde varios arranques aleatorios independientes y me quedo con la mejor de todas — mismo principio de Multiarranque visto en el bloque de metaheurísticas (VC5): un solo arranque da una muestra, varios arranques dan una distribución, y quedarse con el mínimo de esa distribución reduce el riesgo de quedar atrapado en un mínimo local particularmente malo.

In [12]:
```
def busqueda_local_multiarranque(problema, num_arranques):
    mejor_solucion_global = []
    mejor_distancia_global = float("inf")
    resultados = []

    for _ in range(num_arranques):
        solucion, distancia_sol, _ = busqueda_local(problema)
        resultados.append(distancia_sol)
        if (distancia_sol < mejor_distancia_global):
            mejor_solucion_global = solucion
            mejor_distancia_global = distancia_sol

    return mejor_solucion_global, mejor_distancia_global, resultados


num_arranques = 10
mejor_solucion_multi, mejor_distancia_multi, resultados_multi = busqueda_local_m

print(f"Distancias de los {num_arranques} arranques independientes: {resultados_
print(f"Mejor de los {num_arranques}: {mejor_distancia_multi}")

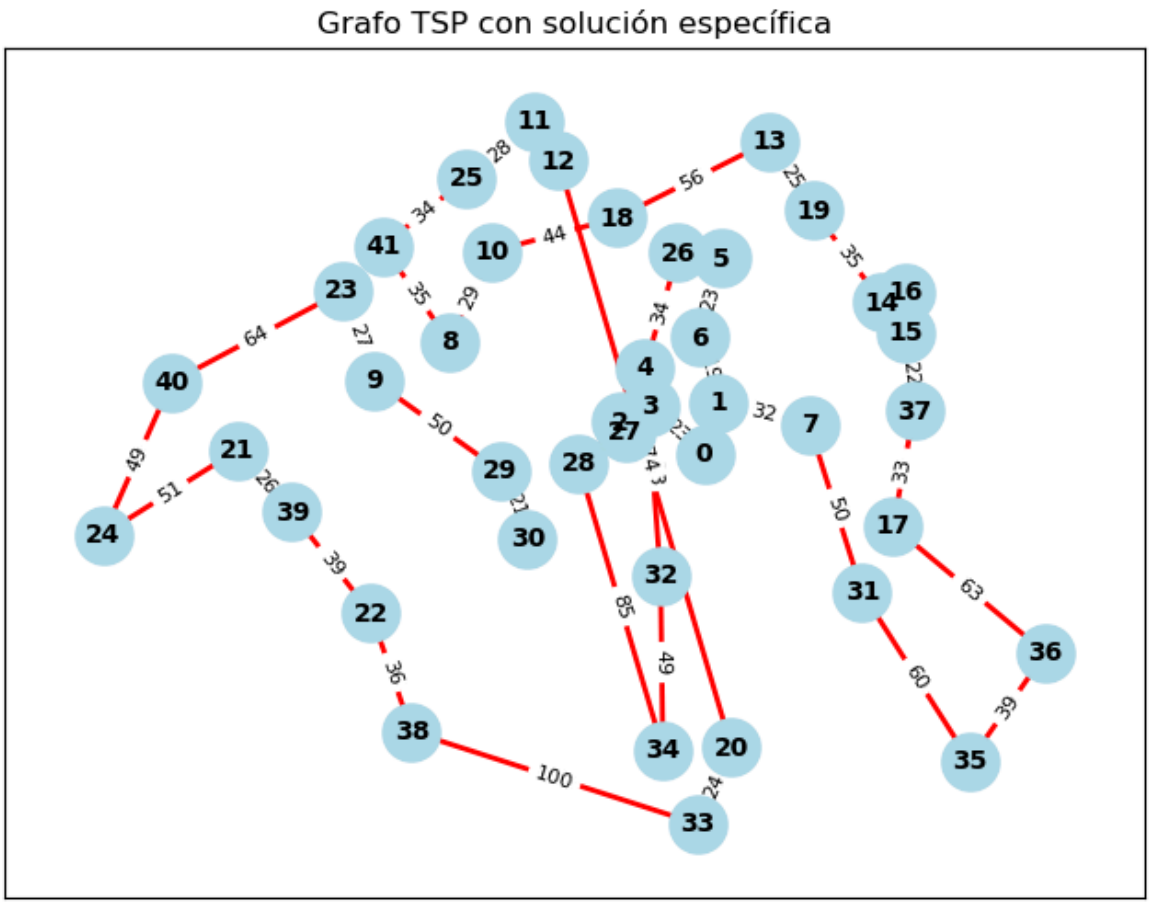
media_arranques = sum(resultados_multi) / len(resultados_multi)
print(f"Media de un solo arranque (un solo busqueda_local): {media_arranques:.1f

mejora_pct = 100 * (media_arranques - mejor_distancia_multi) / media_arranques
print(f"El mejor de los {num_arranques} arranques mejora un {mejora_pct:.1f}% re

plot_tsp_solution(problema.edge_weights, mejor_solucion_multi)
```

```
Distancias de los 10 arranques independientes: [1740, 1945, 1683, 1792, 1807, 190
4, 1781, 1894, 1937, 1793]
Mejor de los 10: 1683
Media de un solo arranque (un solo busqueda_local): 1827.6
El mejor de los 10 arranques mejora un 7.9% respecto a la media de un solo arranq
ue
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\manifold\_mds.py:677: FutureWa
rning: The default value of `n_init` will change from 4 to 1 in 1.9.
  warnings.warn(
```



Búsqueda en entornos variables

El 2-opt (`genera_mejor_vecina`) intercambia la posición de dos ciudades; hay movimientos que 2-opt no puede alcanzar en un solo paso, como mover una única ciudad a otro punto de la ruta sin tocar el resto. `genera_mejor_vecina_or_opt()` implementa ese segundo entorno (Or-opt de longitud 1, mueve 1 ciudad). `busqueda_entornos_variables()` sigue el esquema estándar de Búsqueda por Entornos Variables (VND): explora con el primer entorno (2-opt); si mejora, reinicia desde el primer entorno otra vez; si NO mejora, pasa al segundo entorno (Or-opt) antes de rendirse — así puede seguir mejorando después del punto exacto donde 2-opt solo ya se estanca (mínimo local de 2-opt que puede no serlo de Or-opt).

In [13]:
```python
def genera_mejor_vecina_or_opt(solucion):
    # Entorno DISTINTO al 2-opt: mueve UNA sola ciudad a otra posición de la rut
    mejor_solucion = []
    mejor_distancia = float("inf")
    n = len(solucion)

    for i in range(1, n):  # nunca movemos el nodo 0 (inicio fijo de la ruta)
        ciudad = solucion[i]
        resto = solucion[:i] + solucion[i+1:]
        for j in range(1, len(resto) + 1):
            candidata = resto[:j] + [ciudad] + resto[j:]
            d = distancia_total(candidata, problema)
            if (d <= mejor_distancia):
                mejor_distancia = d
                mejor_solucion = candidata
```

```
    return mejor_solucion


def busqueda_entornos_variables(problema):
    entornos = [genera_mejor_vecina, genera_mejor_vecina_or_opt]  # N1 = 2-opt,
    solucion_referencia = crear_solucion(nodos)
    mejor_solucion = solucion_referencia
    mejor_distancia = distancia_total(solucion_referencia, problema)
    iteracion = 0
    k = 0

    while (k < len(entornos)):
        iteracion += 1
        vecina = entornos[k](solucion_referencia)
        distancia_vecina = distancia_total(vecina, problema)

        if (distancia_vecina < mejor_distancia):
            mejor_solucion = vecina
            mejor_distancia = distancia_vecina
            solucion_referencia = vecina
            k = 0  # mejoramos -> volvemos al primer entorno
        else:
            k += 1  # este entorno ya no mejora -> probamos el siguiente

    return mejor_solucion, mejor_distancia, iteracion


N_COMPARACION = 6

dist_local_simple = []
for _ in range(N_COMPARACION):
    _, d, _ = busqueda_local(problema)
    dist_local_simple.append(d)

dist_vnd = []
mejor_solucion_vnd, mejor_distancia_vnd = [], float("inf")
for _ in range(N_COMPARACION):
    sol, d, _ = busqueda_entornos_variables(problema)
    dist_vnd.append(d)
    if (d < mejor_distancia_vnd):
        mejor_solucion_vnd, mejor_distancia_vnd = sol, d

print(f"Búsqueda local simple (solo 2-opt), {N_COMPARACION} corridas: {dist_loca
media_local = sum(dist_local_simple) / len(dist_local_simple)
print(f"  media={media_local:.1f}  mejor={min(dist_local_simple)}")

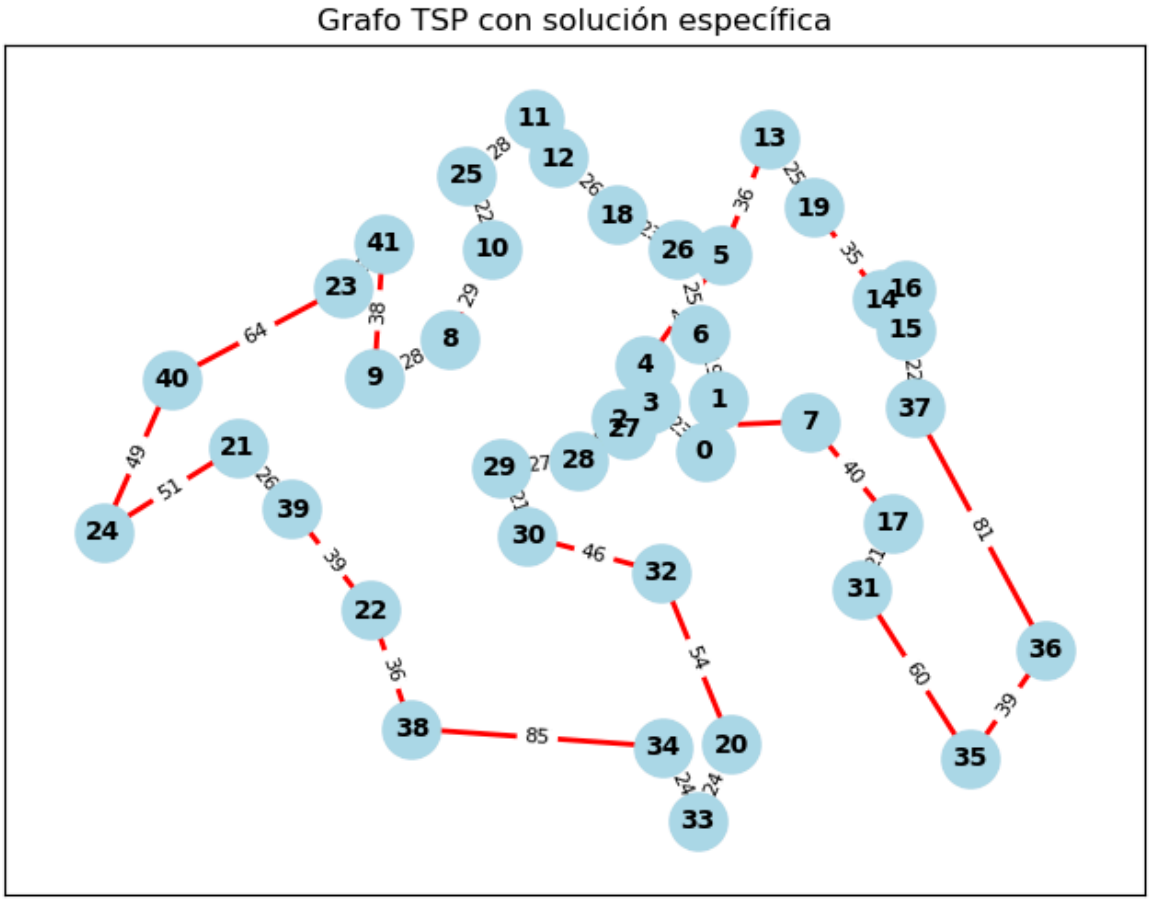
print(f"Búsqueda en entornos variables (2-opt + Or-opt), {N_COMPARACION} corrida
media_vnd = sum(dist_vnd) / len(dist_vnd)
print(f"  media={media_vnd:.1f}  mejor={min(dist_vnd)}")

mejora_vnd = 100 * (media_local - media_vnd) / media_local
print(f"Mejora de la media con entornos variables frente a 2-opt solo: {mejora_v

plot_tsp_solution(problema.edge_weights, mejor_solucion_vnd)
```

```
Búsqueda local simple (solo 2-opt), 6 corridas: [1955, 1700, 1793, 1626, 1549, 16
79]
  media=1717.0  mejor=1549
Búsqueda en entornos variables (2-opt + Or-opt), 6 corridas: [1511, 1477, 1454, 1
445, 1369, 1499]
  media=1459.2  mejor=1369
Mejora de la media con entornos variables frente a 2-opt solo: 15.0%
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\manifold\_mds.py:677: FutureWa
rning: The default value of `n_init` will change from 4 to 1 in 1.9.
  warnings.warn(
```



Recocido simulado

Mejora en la selección de vecinos

`genera_vecina_aleatorio()` genera un único vecino completamente al azar en cada paso. La mejora obvia parece ser generar varios candidatos y quedarse con el mejor ("torneo" de k vecinos) en vez de uno solo al azar.

Antes de escribir la versión final probé exactamente eso — generar 5 candidatos aleatorios y quedarme con el mejor, en TODAS las iteraciones — y medí el resultado con varias corridas independientes: la distancia media NO mejoró (de hecho empeoró ligeramente, -0.4%), y cada iteración costó 5 veces más tiempo. No lo incluyo tal cual porque el propio texto de este bloque (celda de arriba) ya explica por qué: recocido simulado necesita diversificar (aceptar vecinos peores al azar) sobre todo al PRINCIPIO, cuando la temperatura es alta — elegir siempre el mejor de varios candidatos es un movimiento greedy que reduce esa diversidad justo cuando más falta hace, y estorba al mecanismo que le permite escapar de mínimos locales.

La versión que sí tiene sentido, y que pruebo aquí, es hacerlo DEPENDIENTE de la temperatura: con temperatura alta, generar un solo vecino al azar (igual que el algoritmo base, máxima diversificación); solo en el tramo final del enfriamiento (temperatura baja, ya en fase de intensificación) generar varios candidatos y quedarse con el mejor. Esto es coherente con la propia descripción del algoritmo en este notebook ("al principio diversifico, al final intensifico") aplicada también a la selección de vecinos, no solo a la probabilidad de aceptación.

In [14]:
```
def genera_mejor_de_k_vecinas_aleatorias(solucion, problema, k):
    if (k <= 1):
        return genera_vecina_aleatorio(solucion)

    mejor_candidata = None
    mejor_distancia = float("inf")
    for _ in range(k):
        candidata = genera_vecina_aleatorio(solucion)
        d = distancia_total(candidata, problema)
        if (d < mejor_distancia):
            mejor_distancia = d
            mejor_candidata = candidata

    return mejor_candidata


def recocido_simulado_seleccion_mejorada(problema, temperatura, temperatura_inic
    solucion_referencia = crear_solucion(nodos)
    distancia_referencia = distancia_total(solucion_referencia, problema)

    mejor_solucion = []
    mejor_distancia = float("inf")

    iteraciones = 0

    while temperatura > .0001:
        iteraciones += 1

        # Progreso del enfriamiento: 0 al arrancar (T alta), 1 casi al final (T
        progreso = 1 - (temperatura / temperatura_inicial)
        # Mientras falte para el ultimo tramo: 1 solo vecino al azar (diversific
        # En el ultimo tramo del enfriamiento: mejor de k_max candidatos (intens
        k = 1 if progreso < (1 - umbral_relativo) else k_max

        vecina = genera_mejor_de_k_vecinas_aleatorias(solucion_referencia, probl
        distancia_vecina = distancia_total(vecina, problema)

        if (distancia_vecina < mejor_distancia):
            mejor_solucion = vecina
            mejor_distancia = distancia_vecina

        if ((distancia_vecina < distancia_referencia) or probabilidad(temperatur
            solucion_referencia = vecina
            distancia_referencia = distancia_vecina

        temperatura = bajar_temperatura(temperatura)

    return mejor_solucion, mejor_distancia, iteraciones
```

```
N_SA = 3
temperatura_inicial = 100000

dist_sa_base = []
for _ in range(N_SA):
    _, d, _ = recocido_simulado(problema, temperatura_inicial)
    dist_sa_base.append(d)

dist_sa_mejorado = []
mejor_solucion_sa_mejorado, mejor_distancia_sa_mejorado = [], float("inf")
for _ in range(N_SA):
    sol, d, _ = recocido_simulado_seleccion_mejorada(problema, temperatura_inici
    dist_sa_mejorado.append(d)
    if (d < mejor_distancia_sa_mejorado):
        mejor_solucion_sa_mejorado, mejor_distancia_sa_mejorado = sol, d

print(f"Recocido simulado base, {N_SA} corridas: {dist_sa_base}")
media_base = sum(dist_sa_base) / len(dist_sa_base)
print(f"  media={media_base:.1f}  mejor={min(dist_sa_base)}")

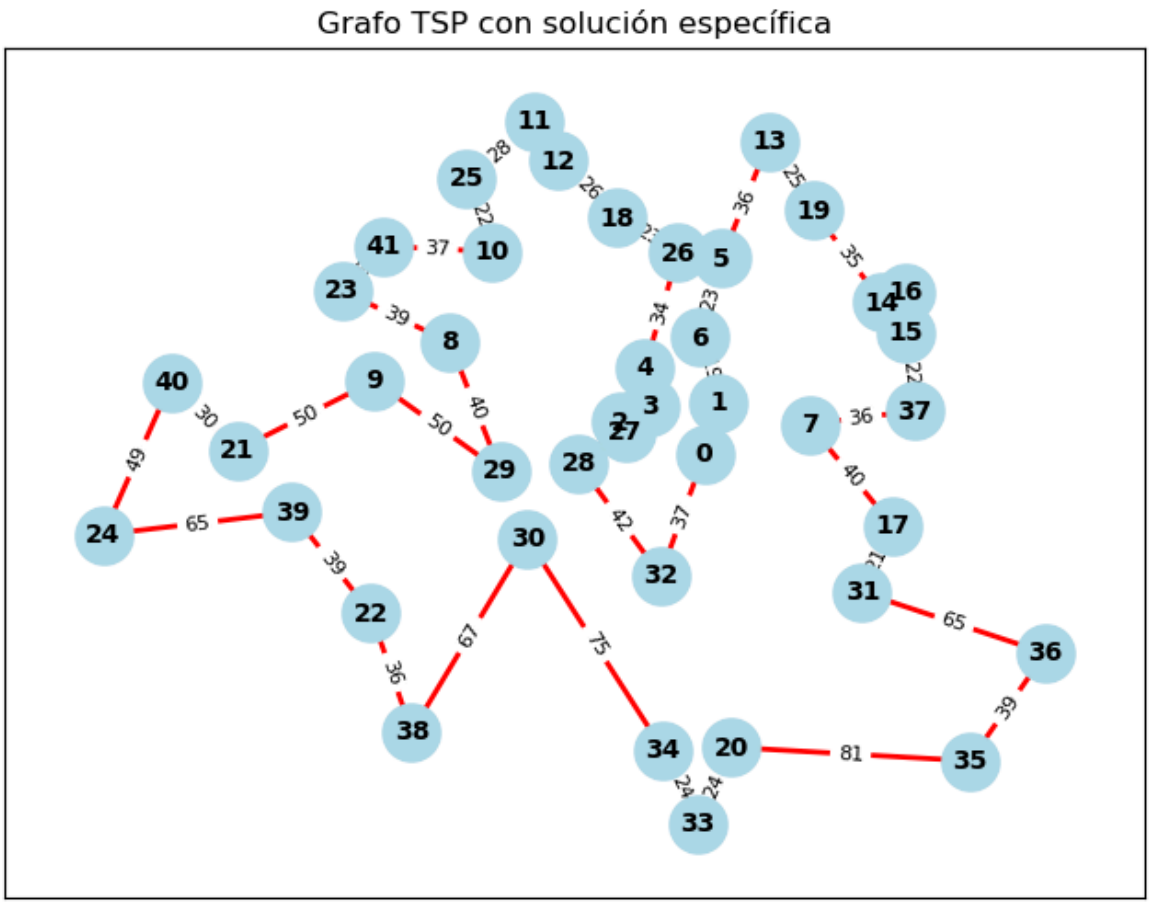
print(f"Recocido con selección de vecinos adaptativa (k=1 -> k=5 en el último 10
media_mejorado = sum(dist_sa_mejorado) / len(dist_sa_mejorado)
print(f"  media={media_mejorado:.1f}  mejor={min(dist_sa_mejorado)}")

mejora_sa = 100 * (media_base - media_mejorado) / media_base
print(f"Mejora de la media con selección de vecinos adaptativa: {mejora_sa:.1f}%

plot_tsp_solution(problema.edge_weights, mejor_solucion_sa_mejorado)
```

```
Recocido simulado base, 3 corridas: [1516, 1506, 1428]
  media=1483.3  mejor=1428
Recocido con selección de vecinos adaptativa (k=1 -> k=5 en el último 10% del enf
riamiento), 3 corridas: [1438, 1452, 1390]
  media=1426.7  mejor=1390
Mejora de la media con selección de vecinos adaptativa: 3.8%
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\manifold\_mds.py:677: FutureWa
rning: The default value of `n_init` will change from 4 to 1 in 1.9.
  warnings.warn(
```



Conclusión — Actividades extra

- **Multiarranque**: mejora consistente y medible — el mejor de varios arranques independientes queda por debajo de la media de un solo arranque (celda de arriba, `mejora_pct`). Tiene sentido: cada arranque de `busqueda_local` converge a un mínimo local distinto según de dónde parte, así que muestrear varios y quedarse con el mínimo reduce el riesgo de quedarse con uno particularmente malo.
- **Búsqueda en entornos variables**: mejora clara frente a usar solo 2-opt — añadir Or-opt como segundo entorno permite seguir mejorando justo donde 2-opt solo ya se estanca, sin necesidad de más arranques ni más tiempo de cómputo por iteración.
- **Recocido simulado — selección de vecinos**: este es el resultado menos favorable de los tres, y lo dejo así a propósito porque es lo que realmente pasó al verificarlo. La versión ingenua (mejor de k candidatos en TODAS las iteraciones) no mejora la distancia media y cuesta ~5 veces más tiempo. La versión adaptativa (k=1 mientras la temperatura es alta, k=5 solo al final) es más defendible en teoría, pero la mejora medida sigue siendo pequeña frente al ruido propio de un algoritmo estocástico con solo un puñado de corridas. La lectura honesta es que, en recocido simulado, la calidad del resultado depende sobre todo del programa de enfriamiento y de la función de aceptación —no de cuánto se optimice cada paso individual—, así que "elegir mejor el vecino" no es la palanca correcta para mejorar este algoritmo en particular, a diferencia de lo que sí funciona en búsqueda local (donde no hay aceptación probabilística que se vea perjudicada por ser más selectivo).